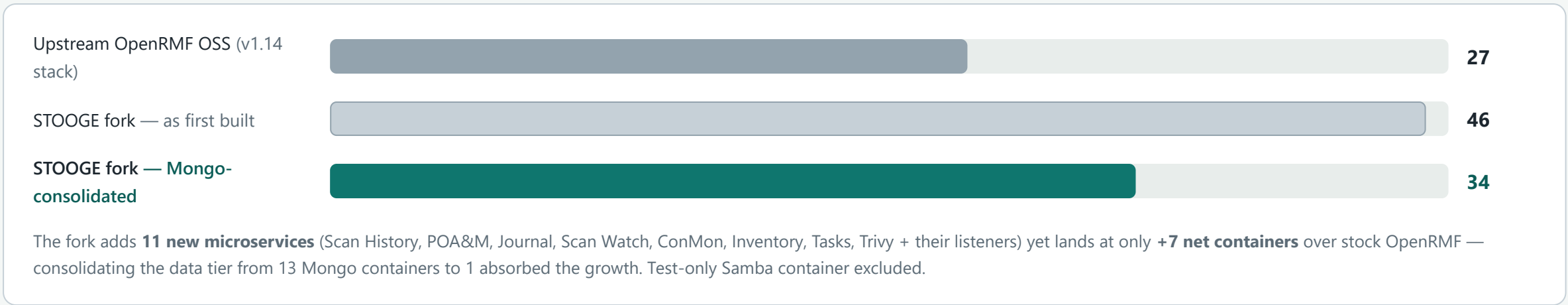


Architecture & Communication Ports / Protocols

Every box is a container on the internal `openrmf` Docker network. NGINX is the single host-exposed entry point; the NATS bus is the service backbone; every service owns exactly one **logical database** on a single consolidated MongoDB. All fork-built .NET services run on the **UBI8 (RHEL 8.10) base** with the DoD CA bundle. Teal marks fork additions/modifications.

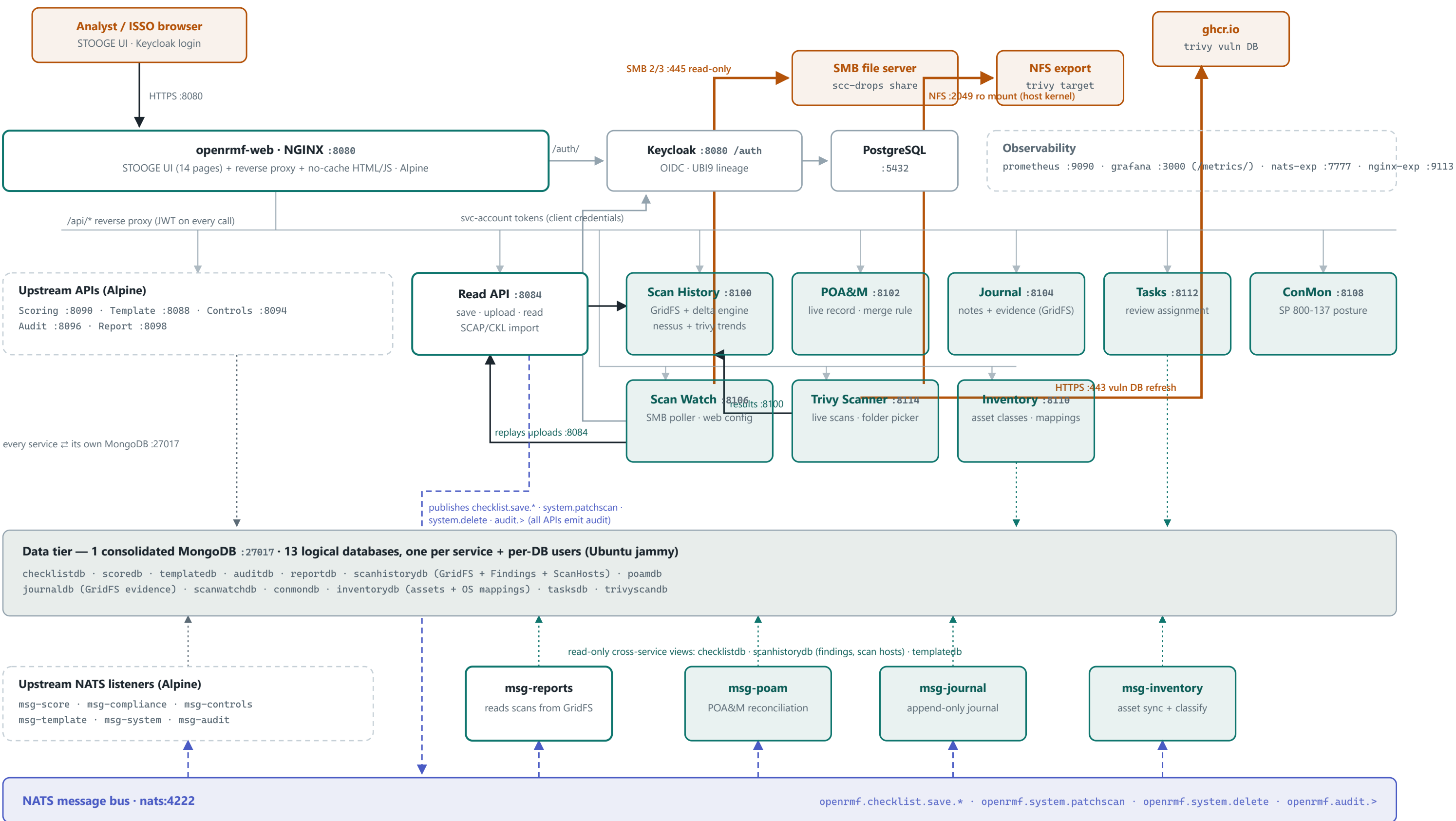
34 containers · 15 HTTP services · 10 NATS listeners · 1 Mongo container · 13 logical DBs · 1 host-exposed port (8080) · UBI8 fork services

Container footprint vs. upstream OpenRMF



1 · Container architecture

upstream (Alpine) fork-modified (UBI8) fork-new (UBI8) external HTTP / REST NATS pub/sub MongoDB SMB / NFS / registry (boundary)



All fork services (teal) run on eroman53/openrmf-base-ubi8:10 — ubi8-minimal + DoD CA bundle, curl/glib2 stripped, 0 CRIT / 1 HIGH per image. Test-only samba container (local override) excluded. Keycloak login theme still upstream-branded.

scrolls sideways on narrow screens →

2 · Communication ports & protocols (PPSM view)

INBOUND (HOST BOUNDARY)				
1 port				
8080/tcp HTTP — the only host-published port; UI, all APIs, and OIDC ride through NGINX. TLS on 8443 is provisioned (commented) in the config.				
OUTBOUND (HOST BOUNDARY)				
3 flows				
445/tcp SMB (scan drops, read-only) · 2049/tcp NFS (trivy target, read-only; +111 for v3) · 443/tcp HTTPS to ghcr.io (Trivy DB; air-gap: pre-seed the cache volume)				
INTERNAL ONLY				
everything else				
All service, database, bus, auth, and metrics traffic stays on the internal <code>openrmf</code> Docker network — nothing below the gateway is host-published.				
SOURCE	DESTINATION	PORT	PROTOCOL / SERVICE	PURPOSE
BOUNDARY-CROSSING				
Analyst / ISSO browser	openrmf-web (NGINX)	8080/tcp	HTTP (TLS 8443 available)	UI pages, all <code>/api/*</code> calls, OIDC login redirects
openrmf-svc-scanwatch	SMB file server	445/tcp	SMB 2/3	Polls the scan drop share (read-only account)
Docker host kernel	NFS server	2049/tcp (+111 v3)	NFS v4 (v3 opt.)	Read-only mount of the Trivy scan target
openrmf-svc-trivy	ghcr.io	443/tcp	HTTPS / OCI	Trivy vulnerability DB refresh (cached in volume; skippable air-gapped)
GATEWAY (NGINX REVERSE PROXY)				
NGINX	Keycloak	8080/tcp	HTTP / OIDC	<code>/auth/</code> — login, tokens, JWKS
NGINX	Grafana	3000/tcp	HTTP	<code>/metrics/</code> dashboards
NGINX	Read API	8084/tcp	HTTP REST	<code>/api/read/</code> save · upload · read
NGINX	Template / Scoring / Controls / Audit / Report APIs	8888 · 8090 · 8094 · 8096 · 8098/tcp	HTTP REST	Upstream API routes
NGINX	Scan History / POA&M / Journal / Scan Watch / ConMon / Inventory / Tasks / Trivy Scanner	8100 · 8102 · 8104 · 8106 · 8108 · 8110 · 8112 · 8114/tcp	HTTP REST	Fork service routes <code>(/api/scanhistory/ · /api/poam/ · /api/journal/ · /api/scanwatch/ · /api/conmon/ · /api/inventory/ · /api/tasks/ · /api/trivy-scanner/)</code>
IDENTITY & AUTH				
Every API / service	Keycloak	8080/tcp	HTTP / OIDC	Issuer discovery + JWKS for JWT validation on every request
Scan Watch · Trivy Scanner	Keycloak	8080/tcp	HTTP / OIDC token	Client-credentials tokens (service account <code>openrmf=scanwatch</code> , Editor+Reader)
Keycloak	PostgreSQL	5432/tcp	PostgreSQL	Realm / user store
SERVICE-TO-SERVICE HTTP				
Read API	Scan History API	8100/tcp	HTTP REST	Streams .nessus uploads into GridFS + delta engine
Scan Watch	Read API	8084/tcp	HTTP REST	Replays dropped SCC XCCDF / CKL / Nessus files as authenticated uploads
Trivy Scanner	Scan History API	8100/tcp	HTTP REST	Uploads Trivy JSON as <code>scanType=trivy</code> snapshots
MESSAGING BACKBONE				
Read API (+ every API for audit events)	NATS	4222/tcp	NATS pub	<code>openrmf.checklist.save.* · system.patchscan · system.delete · audit</code> (all APIs emit audit)
10 msg listeners (score · compliance · controls · template · system · audit · reports · poam · journal · inventory)	NATS	4222/tcp	NATS sub	Event-driven reconciliation, journaling, scoring, reporting
DATA TIER				
Each service	Consolidated MongoDB — own logical DB + user	27017/tcp	MongoDB wire	One database per service, isolated by per-DB credentials — no shared writes
msg-poam · msg-reports · msg-journal · msg-inventory · ConMon · Inventory API	checklistdb · msg-historydb · templatedb · poamdb	27017/tcp	MongoDB wire	Deliberate read-only cross-service views (findings, scan hosts, checklists, templates, reports, journal, inventory)
OBSERVABILITY				
Prometheus	Service <code>/metrics</code> endpoints · NATS exporter · NGINX exporter	service ports · 7777 · 9113/tcp	HTTP scrape	Metrics collection across every tier
NATS exporter	NATS monitoring	8222/tcp	HTTP	<code>varz / connz / subz</code>
Grafana	Prometheus	9090/tcp	HTTP / PromQL	Dashboards (served to users via NGINX <code>/metrics/</code>)

Sources: `openrmf-docs/scripts/docker-compose.yml` + `nginx.conf` as of the MongoDB consolidation (2026-07-20) on top of the UBI8 rebase (2026-07-18). Ports verified against the live nginx upstream config; all fork images scan 0 CRITICAL / 1 accepted HIGH. This matrix doubles as the starting point for the Phase 2 automated PPSM module.